Tab 1

Track 1	Classical ML (Regression & Clustering) - Machine Learning approach advised by Noah (seminar 1)

Task 1 (Business Analytic Question)

Each group must propose a business-oriented question relevant to Airbnb. The question should be practical and valuable to Airbnb's strategic planning. An example might be: "How do seasonal fluctuations affect Airbnb pricing strategies in different neighbourhoods?"

Task 2 (Data Analysis – Based on Track Selection)

Each group must apply their assigned machine learning approach to analyse the dataset.

Track-Specific Expectations:

- Track 1 (Classical ML – Regression & Clustering): Apply regression models for pricing prediction and clustering techniques for customer segmentation.

steps by Claude

Claude

Task 1: Business question
"Which listing, host, and location characteristics drive Airbnb pricing across NYC neighbourhoods, and can listings be segmented into distinct groups to support differentiated pricing and marketing strategy?"

Task 2 — Data analysis plan:

1. Load & inspect data (types, missing values, duplicates)
2. Clean data (drop irrelevant cols, handle missing reviews_per_month/last_review, remove price=0 and extreme outliers)
3. EDA (price distribution, price by borough/room type, correlation heatmap, geospatial price map)
4. Feature engineering (log-price target, one-hot encode categoricals, scale numerics)
5. Regression — predict price with Linear Regression, Ridge, and Random Forest; compare R^2/RMSE/MAE; extract feature importance
6. Clustering — KMeans segmentation on listing behavior (price, availability, reviews, minimum nights); pick k via elbow/silhouette; profile & name segments
7. Tie regression + clustering findings together

Task 3 — Deliverable:
Analytical report (≤1000 words) with visuals, written for Airbnb executives.

—----------

analysis.py — the complete, executable pipeline covering every step:

- Load & inspect (48,895 rows)
- Clean (drop $0-price and top 1% outliers → 48,396 rows)
- EDA (price distribution, borough/room-type comparisons, geospatial map, correlation heatmap)
- Feature engineering (log-price target, one-hot encoding, neighbourhood frequency encoding)
- Regression — Linear, Ridge, Random Forest for price prediction
- Clustering — KMeans (k=4, chosen via elbow/silhouette) for listing segmentation

Key results:

- Random Forest beat linear models (R^2=0.63 vs 0.52; MAE ≈ $43 vs $49) — room type and location dominate price.
- Four segments emerged: High-Turnover Favorites (15%), Long-Stay/Portfolio Hosts (24%), Premium Professional Operators (<1% but ~288 listings/host), and Casual/Occasional Hosts (59%, the long tail).

Airbnb_NYC_2019_Executive_Report.docx — the ≤1000-word Task 3 deliverable, written for Airbnb's executive team with the key charts embedded and strategic recommendations tied to the ML findings.

All supporting figures, the cluster profile table, and raw regression metrics are also included if you want to dig into specifics or swap them into your own slides.

report 1kw by Claude

Airbnb NYC 2019: Pricing Drivers & Listing Segmentation

Analytical Report for Airbnb Executive Team | Track 1 — Classical ML (Regression & Clustering)

Business Question

Which listing, host, and location characteristics drive Airbnb pricing across New York City's neighbourhoods, and can listings be segmented into distinct groups to support differentiated pricing and marketing strategy? Answering this helps Airbnb calibrate host pricing guidance, target marketing spend, and identify professional operators shaping local supply.

Data & Method

We analysed 48,895 NYC listings from the 2019 Inside Airbnb dataset (host, location, room type, pricing, and availability fields). After removing invalid records (zero-price listings and the top 1% price outliers) and handling missing review data, 48,396 listings remained. Two classical ML techniques were applied:

- Regression - Linear, Ridge, and Random Forest models trained on log-transformed price to identify pricing drivers and benchmark predictive accuracy.
- Clustering - K-Means (k=4, selected via elbow and silhouette analysis) on price, availability, review activity, and host portfolio size to segment listings into behavioural groups.

Finding 1 — What Drives Price

Price varies sharply by borough and room type. Manhattan commands the highest median price, followed by Brooklyn, while the Bronx and Staten Island are markedly cheaper. Entire homes/apartments price well above private or shared rooms.

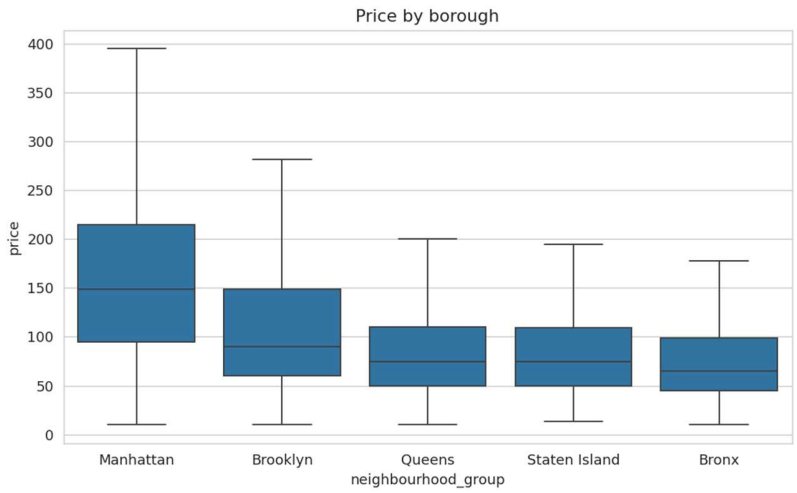


Figure 1. Listing price by borough (outliers excluded for readability).

Random Forest outperformed linear models, capturing non-linear location and availability effects:

Model	R^2 (log scale)	RMSE ($)	MAE ($)
Linear Regression	0.52	$85.55	$48.98
Ridge Regression	0.52	$85.55	$48.98
Random Forest	0.63	$75.30	$43.08

Feature importance confirms room type and borough/neighbourhood popularity as the dominant price drivers, with availability and host portfolio size contributing secondary signals.

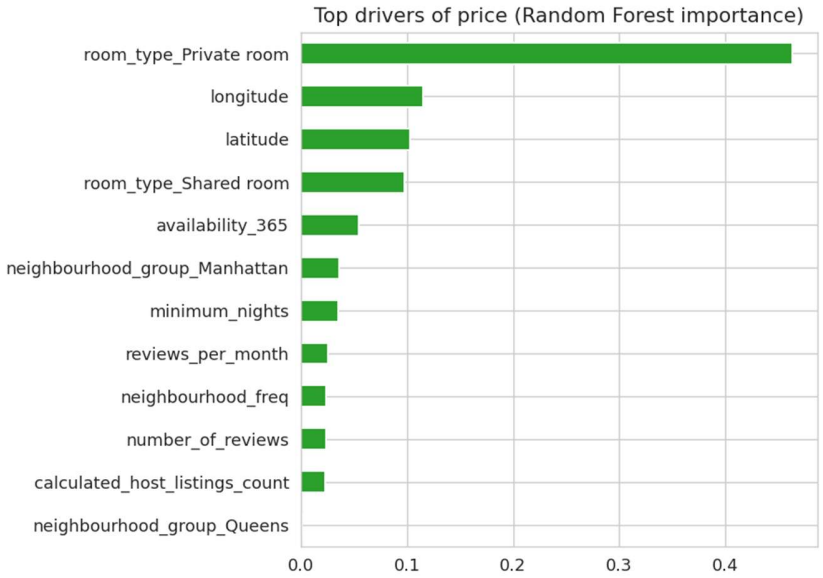


Figure 2. Top predictors of price (Random Forest feature importance).

Finding 2 — Four Distinct Listing Segments

Clustering on booking and availability behaviour (not just price) reveals four operationally distinct segments:

Segment	Avg Price	Avg Min. Nights	Avg Reviews	Avg Availability	Listings/Host	Share of Listings

High-Turnover Favorites	$120	2.5	97.8	156	2.1	15,00%
Long-Stay / Portfolio Hosts	$170	14.5	13.7	290	10.6	24,00%
Premium Professional Ops.	$274	21.2	2.3	282	287.6	1,00%
Casual / Occasional Hosts	$126	4.5	8.6	26	1.7	59,00%

- High-Turnover Favorites (15%): budget-friendly, short minimum stays, very high review volume — Airbnb's most active, guest-loved inventory.
- Long-Stay / Portfolio Hosts (24%): mid-priced, longer minimum stays, hosts managing ~11 listings each — a semi-professional segment optimising for occupancy.
- Premium Professional Operators (1% of listings, but highest price and ~288 listings per host): large-scale, business-style operators — a small but commercially significant group.
- Casual / Occasional Hosts (59%, the majority): low availability, low review activity, single-listing hosts — the long tail of the marketplace.

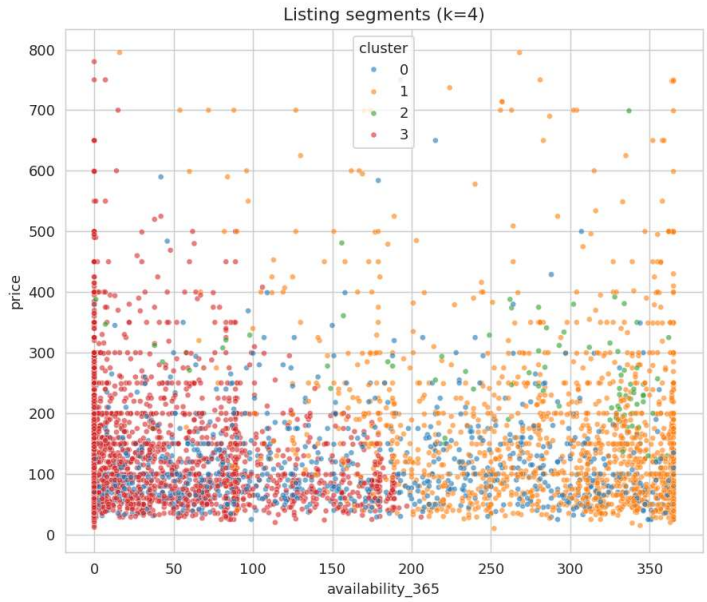


Figure 3. Listing segments by price and annual availability.

How the ML Approach Shaped the Results

Regression quantified which attributes move price, while clustering revealed that listings with similar prices can have very different operating models — for example, Long-Stay/Portfolio Hosts and Casual Hosts have similar median prices ($170 vs $126) but opposite availability and scale profiles. Combining both techniques gives a fuller picture than either alone: regression supports fair, data-driven pricing guidance, while clustering identifies who is behind the supply, which matters for policy, support, and marketing decisions.

Strategic Recommendations

- Use the Random Forest price model (or a refined version) to power a 'suggested price' tool for hosts, prioritising room type, neighbourhood, and availability inputs.
- Target Casual/Occasional Hosts (59% of supply) with onboarding nudges to raise availability and review activity — the largest lever for overall marketplace liquidity.
- Engage Premium Professional Operators separately via account management, given their outsized scale despite being <1% of listings.
- Monitor Long-Stay/Portfolio Hosts for extended-stay product opportunities, given their longer minimum-night patterns.

Limitations

This analysis uses a single-year snapshot (2019) with no seasonality data, and neighbourhood-level effects were approximated via frequency encoding rather than full categorical modelling. Regression R^2 (~0.63) indicates room for improvement with richer features (e.g., amenities, photos, text descriptions) in future iterations.

py by Claude

```
# ------------------------------------------------------------------------
# STEP 1: LOAD & INSPECT
# ------------------------------------------------------------------------
df = pd.read_csv("/home/claude/AB_NYC_2019.csv")
print("Raw shape:", df.shape)

# ------------------------------------------------------------------------
# STEP 2: CLEAN
# ------------------------------------------------------------------------
df = df.drop(columns=["id", "name", "host_name", "last_review"])

# reviews_per_month missing => listing has 0 reviews => fill with 0
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

# remove price outliers: 0 (invalid) and top 1% (extreme luxury noise)
df = df[df["price"] > 0]
upper = df["price"].quantile(0.99)
df = df[df["price"] <= upper]

# minimum_nights sanity cap (remove absurd values e.g. >365)
df = df[df["minimum_nights"] <= 365]

print("Clean shape:", df.shape)

# ------------------------------------------------------------------------
# STEP 3: EDA
# ------------------------------------------------------------------------
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.histplot(df["price"], bins=50, ax=axes[0], color="#1f77b4")
axes[0].set_title("Price distribution")
sns.histplot(np.log1p(df["price"]), bins=50, ax=axes[1], color="#ff7f0e")
axes[1].set_title("Log(price+1) distribution")
plt.tight_layout()
plt.savefig(f"{OUT}/price_distribution.png", dpi=130)
plt.close()

plt.figure(figsize=(8, 5))
order =
df.groupby("neighbourhood_group")["price"].median().sort_values(ascending=False).index
sns.boxplot(data=df, x="neighbourhood_group", y="price", order=order,
showfliers=False)
plt.title("Price by borough")
plt.tight_layout()
plt.savefig(f"{OUT}/price_by_borough.png", dpi=130)
plt.close()

plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="room_type", y="price", showfliers=False)
plt.title("Price by room type")
plt.tight_layout()
plt.savefig(f"{OUT}/price_by_room_type.png", dpi=130)
plt.close()

plt.figure(figsize=(7, 6))
sc = plt.scatter(df["longitude"], df["latitude"], c=df["price"], cmap="viridis",
```

```
                s=3, alpha=0.5, vmax=df["price"].quantile(0.95))
plt.colorbar(sc, label="price ($)")
plt.title("Geospatial price map - NYC listings")
plt.xlabel("longitude"); plt.ylabel("latitude")
plt.tight_layout()
plt.savefig(f"{OUT}/geo_price_map.png", dpi=130)
plt.close()

num_cols = ["price", "minimum_nights", "number_of_reviews", "reviews_per_month",
            "calculated_host_listings_count", "availability_365"]
plt.figure(figsize=(7, 6))
sns.heatmap(df[num_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation matrix")
plt.tight_layout()
plt.savefig(f"{OUT}/correlation_heatmap.png", dpi=130)
plt.close()

# ---------------------------------------------------------------------
# STEP 4: FEATURE ENGINEERING
# ---------------------------------------------------------------------
model_df = df.copy()
model_df["log_price"] = np.log1p(model_df["price"])

cat_cols = ["neighbourhood_group", "room_type"]
model_df = pd.get_dummies(model_df, columns=cat_cols, drop_first=True)

# neighbourhood has too many levels for OHE -> target-encode with mean log price
(train-safe via CV would be ideal;
# for simplicity/interpretability here we use frequency encoding as a proxy signal)
neigh_freq = model_df["neighbourhood"].value_counts()
model_df["neighbourhood_freq"] = model_df["neighbourhood"].map(neigh_freq)
model_df = model_df.drop(columns=["neighbourhood"])

feature_cols = [c for c in model_df.columns if c not in ["price", "log_price",
"host_id"]]

# ---------------------------------------------------------------------
# STEP 5: REGRESSION - PRICE PREDICTION
# ---------------------------------------------------------------------
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

X = model_df[feature_cols]
y = model_df["log_price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

```
results = {}

lr = LinearRegression().fit(X_train_s, y_train)
ridge = Ridge(alpha=5.0).fit(X_train_s, y_train)
rf = RandomForestRegressor(n_estimators=300, max_depth=12, random_state=42, n_jobs=-
1).fit(X_train, y_train)

for name, model, Xte in [("Linear Regression", lr, X_test_s),
                         ("Ridge Regression", ridge, X_test_s),
                         ("Random Forest", rf, X_test)]:
   pred_log = model.predict(Xte)
   pred = np.expm1(pred_log)
   actual = np.expm1(y_test)
   results[name] = {
       "R2_logscale": round(r2_score(y_test, pred_log), 4),
       "RMSE_dollars": round(np.sqrt(mean_squared_error(actual, pred)), 2),
       "MAE_dollars": round(mean_absolute_error(actual, pred), 2),
   }

print(json.dumps(results, indent=2))

# feature importance (Random Forest)
importances = pd.Series(rf.feature_importances_,
index=feature_cols).sort_values(ascending=False).head(12)
plt.figure(figsize=(7, 5))
importances.sort_values().plot(kind="barh", color="#2ca02c")
plt.title("Top drivers of price (Random Forest importance)")
plt.tight_layout()
plt.savefig(f"{OUT}/feature_importance.png", dpi=130)
plt.close()

# ---------------------------------------------------------------------------
# STEP 6: CLUSTERING - LISTING SEGMENTATION
# ---------------------------------------------------------------------------
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

cluster_features = ["price", "minimum_nights", "number_of_reviews",
                    "reviews_per_month", "availability_365",
"calculated_host_listings_count"]
Xc = df[cluster_features].copy()
Xc_scaled = StandardScaler().fit_transform(Xc)

inertias, sils = [], []
K_range = range(2, 9)
for k in K_range:
   km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(Xc_scaled)
   inertias.append(km.inertia_)
   sils.append(silhouette_score(Xc_scaled, km.labels_))

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(list(K_range), inertias, marker="o")
plt.title("Elbow method"); plt.xlabel("k"); plt.ylabel("inertia")
plt.subplot(1, 2, 2)
```

```
plt.plot(list(K_range), sils, marker="o", color="orange")
plt.title("Silhouette score"); plt.xlabel("k"); plt.ylabel("score")
plt.tight_layout()
plt.savefig(f"{OUT}/cluster_selection.png", dpi=130)
plt.close()

best_k = 4  # chosen from elbow/silhouette inspection
km = KMeans(n_clusters=best_k, random_state=42, n_init=10).fit(Xc_scaled)
df["cluster"] = km.labels_

profile = df.groupby("cluster")[cluster_features].mean().round(1)
profile["count"] = df.groupby("cluster").size()
print(profile)
profile.to_csv(f"{OUT}/cluster_profile.csv")

plt.figure(figsize=(7, 6))
sns.scatterplot(data=df.sample(6000, random_state=1), x="availability_365", y="price",
                hue="cluster", palette="tab10", s=15, alpha=0.6)
plt.title(f"Listing segments (k={best_k})")
plt.tight_layout()
plt.savefig(f"{OUT}/cluster_scatter.png", dpi=130)
plt.close()

# save metrics for report writing
with open(f"{OUT}/regression_results.json", "w") as f:
   json.dump(results, f, indent=2)

print("DONE")
```

Tab 5

How does distance from Time Square affect the price of Airbnb in NYC?

17th august

1. Data preparation (Jeroen)

21st august
Write down already some of the process for the final report

2. Regression (Damiano)
3. Clustering (Adaeze)

27th go august

4. Final report (Sarah)

1-2nd september

Emergency 3-6th september